

AiMitra

Your Intelligent AI Assistant

Empowering QA Engineers, Business Analysts & Developers
to build better software, faster.

13

AI Providers

50+

AI Models

30+

Specialised Tools

4

Workspaces

Chat & Code

Dev Tools

JIRA Tools

BA Tools

[Complete Application Documentation](#) | [All Tools](#) | [Demo Data](#) | [Getting Started](#)

Developed by Shiv Kant Kumar

linkedin.com/in/shivkantkumar/

Table of Contents

1.	About AiMitra	<i>Overview, vision, and what makes it unique</i>
2.	Tech Stack	<i>Frontend, backend, AI integration, and infrastructure</i>
3.	Unique Selling Points (USP)	<i>Why AiMitra stands out from other tools</i>
4.	Getting Started	<i>Installation, setup, and first use</i>
5.	Chat & Code Workspace	<i>7 AI-powered chat modes with demo data</i>
5.1	Text / Code Generation	
5.2	Code Debugging	
5.3	Test Case Generator	
5.4	Test Plan Generator	
5.5	DOM Locator Generator	
5.6	Web Search	
5.7	Image Generation	
6.	Dev Tools Workspace	<i>14 specialised developer tools</i>
6.1-6.14	All Dev Tools with demo data	
7.	JIRA Tools Workspace	<i>7 JIRA-integrated tools</i>
7.1-7.7	All JIRA Tools with demo data	
8.	BA Tools Workspace	<i>10 Business Analyst tools</i>
8.1-8.10	All BA Tools with demo data	
9.	Chrome Extension	<i>Direct DOM analysis & test generation in browser</i>
10.	AI Providers & Models	<i>All 13 providers and 50+ models</i>
11.	Architecture & Data Flow	<i>How the app works end-to-end</i>
12.	About the Developer	<i>Shiv Kant Kumar -- contact & LinkedIn</i>

1. About AiMitra

AiMitra ("AI Friend" in Sanskrit) is a fully-featured, open-architecture AI assistant designed specifically for the three pillars of software delivery: Quality Assurance (QA), Business Analysis (BA), and Software Development. It consolidates more than 30 purpose-built AI tools into a single, beautifully designed application, eliminating the need to juggle multiple AI services.

Unlike generic AI chat tools, every workspace and every tool inside AiMitra has a carefully crafted system prompt that turns the AI into a domain expert -- a senior QA lead, a seasoned BA, or a code reviewer -- depending on what you need. The smart mode detector even suggests the right tool automatically when it detects keywords in your message.

2. Tech Stack

Frontend

React 18.3	UI framework with hooks, context, and automatic batching
Tailwind CSS	Utility-first styling with custom dark theme (no CSS files)
React Markdown + remark-gfm	Renders AI responses as rich markdown with tables, code blocks
React Syntax Highlighter	VS Code Dark+ themed code blocks with copy button
Axios + Fetch API	Axios for non-streaming calls; native Fetch for SSE streaming
localStorage	Persists API keys, config, conversation history -- no account needed

Backend

FastAPI 0.115	High-performance async Python web framework; OpenAPI docs auto-generated
Uvicorn	Lightning-fast ASGI server; serves both API and built React app
httpx	Fully async HTTP client for all AI provider API calls
Pydantic v2	Request/response validation with strict typing
SSE-Starlette	Server-Sent Events for real-time token streaming
BeautifulSoup4 + lxml	HTML parsing for the web page DOM analyser
Supabase	Optional auth/database integration

AI Integration

AiMitra supports 13 AI providers (OpenAI, Anthropic, Google Gemini, Groq, Mistral, DeepSeek, xAI/Grok, Together AI, Perplexity, Cerebras, OpenRouter, Fireworks AI, Cohere) and 50+ individual models. The backend acts as a thin proxy: API keys are entered by the user, stored in localStorage, and forwarded per-request -- they are never stored on the server. Provider routing is automatic based on the selected model/provider. All providers support both streaming (SSE) and non-streaming modes.

3. Unique Selling Points (USP)

* All-in-One Platform

30+ specialised tools across 4 workspaces (Chat, Dev Tools, JIRA, BA). No context switching between multiple AI services. Everything a QA, BA, and Dev needs in one interface.

* No Vendor Lock-in -- 13 Providers

Switch between OpenAI, Gemini, Groq (free), Anthropic, DeepSeek and 9 more at any time. Free-tier providers (Groq, Gemini, Cerebras, OpenRouter) mean you can use the full feature set at zero cost.

* Domain-Expert AI (Specialised Prompts)

Each tool has a meticulously crafted system prompt that transforms the AI into a domain expert: senior QA lead for test plans, seasoned BA for BRDs, expert debugger for code issues. Generic AI gives generic results -- AiMitra gives expert results.

* Smart Mode Detector

Paste a stack trace in the wrong mode? AiMitra detects it and suggests Code Debugging. Ask for test cases in chat? It suggests Test Case Generator. The smart detector uses keyword analysis with no extra API calls.

* Real-time Streaming

Every AI response streams token-by-token to the UI. No waiting for the full response -- you start reading as the AI thinks. Switchable off for providers that don't support it.

* Chrome Extension

The built-in Chrome extension reads the live DOM of any webpage, captures real CSS selectors, XPath, form structures, and feeds them directly to AI -- generating test code grounded in the actual page, not generic placeholders.

* Conversation History

Every chat and tool output is auto-saved to local history with timestamp, model, and title. Resume any previous conversation with a single click. History is private -- stored in your browser only.

* Zero Infrastructure Cost

No database required. No user accounts. No monthly subscription. Deploy on any free hosting (Render, Railway, Vercel) -- or run locally with two terminal commands.

* BA Tools -- Often Overlooked

Most AI tools focus on developers. AiMitra's dedicated BA workspace with User Story Generator, Gap Analysis, BRD Generator, Impact Analysis, and 6 more tools makes it the only QA/dev AI tool that also fully serves business analysts.

* Comparison with alternatives

vs. ChatGPT/Claude.ai: no specialised QA/BA prompts, no JIRA integration, no Chrome extension.

vs. GitHub Copilot: code only, no QA, no BA, no test planning.

vs. Testim/Mabl: expensive, no BA tools, no multi-provider AI.

AiMitra: free-tier capable, multi-role, multi-provider, all-in-one.

4. Getting Started

Prerequisites

- * Node.js 18+
- * Python 3.11+
- * Git
- * A free API key from Groq (console.groq.com) or Google AI Studio

Installation -- Local Development

Step 1 -- Clone the repository

```
git clone https://github.com/shivkantkumar/aimitra.git
cd aimitra
```

Step 2 -- Backend setup

```
cd backend
pip install -r requirements.txt
cp .env.example .env # add your keys if needed
uvicorn main:app --reload --port 8000
```

Step 3 -- Frontend setup (new terminal)

```
cd frontend
npm install
npm start # opens http://localhost:3000
```

Step 4 -- Add your API key

Open the app in browser -> Sidebar -> Provider: Groq -> API Key: paste key -> Save

First Use Walkthrough

1. Select a Provider in the left sidebar (Groq is free -- no credit card required).
2. Paste your API key and press Enter. A green checkmark confirms it is saved.
3. The default workspace is Chat. Type any message and press Enter or click Send.
4. Try: "Write test cases for a login page" -- the smart detector will suggest switching to Test Case Generator.
5. Switch to Dev Tools (top nav) to access Code Review, BDD Generator, API Test Generator, and more.
6. Switch to BA Tools to access User Story Generator, BRD Generator, Gap Analysis, and more.
7. Switch to JIRA to create tickets, validate stories, or generate JQL queries.
8. Install the Chrome Extension from the chrome-extension/ folder to generate tests from any webpage.

Configuration Options

Provider	Choose from 13 AI providers. Switch at any time -- conversation continues.
Model	Select any model from the chosen provider's list.
Temperature	0.0 = precise/deterministic. 1.0 = creative/varied. Default: 0.7.
Streaming	Toggle real-time token streaming. Disable for providers with latency issues.
API Key	Stored in browser localStorage. Never sent to any server except the chosen AI provider.

5. Chat & Code Workspace

The Chat workspace is the heart of AiMitra. It provides 7 specialised modes, each with a tailored AI system prompt. The Smart Mode Detector automatically suggests the best mode based on your message. All conversations are auto-saved to Activity History.

? Text / Code Generation

When: Everyday use -- answering questions, generating code, writing documentation, explaining concepts.
Why: General-purpose mode with a senior engineer persona. Provides structured, well-explained answers.
How: Select 'Text/Code Generation' from sidebar -> type your question -> press Enter.

Demo Input -- paste this to test:

Write a Python function to validate email addresses using regex. Add type hints and docstring.

Expected: Returns a clean Python function with type hints, docstring, regex pattern, and usage example.

? Code Debugging

When: When you have an error, stack trace, or broken code that needs fixing.
Why: The AI acts as an expert debugger -- identifies root cause, explains the bug, and provides a corrected fix.
How: Paste your code + the error message -> select Code Debugging mode -> submit.

Demo Input -- paste this to test:

TypeError: unsupported operand type(s) for +: 'int' and 'str'
Line 15: result = count + " items"
Fix this.

Expected: Identifies type mismatch, explains the root cause, provides fixed code with str(count) + " items".

? Test Case Generator

When: When you need Selenium/Playwright test cases for a feature, user story, or API endpoint.
Why: Generates production-quality tests with proper assertions, waits, POM structure, and edge cases.
How: Describe the feature or paste the user story -> select Test Case Generator -> submit.

Demo Input -- paste this to test:

Generate Playwright Python test cases for a login page with username, password fields and a submit button. Include happy path, wrong password, empty fields, and SQL injection tests.

Expected: Returns complete Playwright test file with fixtures, parametrize decorators, and 4 test functions.

? Test Plan Generator

When: At the start of a sprint or feature cycle when you need a formal QA test plan.
Why: Creates IEEE 829-compliant test plans with scope, risks, entry/exit criteria, and resource planning.
How: Select Test Plan Generator -> describe the feature/project -> submit.

Demo Input -- paste this to test:

Create a test plan for an e-commerce checkout flow: cart, address, payment (Stripe), and order confirmation. API + UI testing required.

Expected: Returns a structured test plan with objectives, scope, test types, risks, schedule, and sign-off section.

? DOM Locator Generator

When: When writing automation scripts and you need reliable XPath/CSS selectors for a web element.
Why: Generates robust, stable locators using ID, data-testid, aria-label -- not fragile nth-child paths.
How: Describe the element or paste the HTML snippet -> select DOM Locator Generator -> submit.

Demo Input -- paste this to test:

Generate locators for: <button class="btn-primary" id="submit-order" aria-label="Place Order">Place Order</button>

Expected: Returns CSS selector, XPath, and Playwright/Selenium code snippets using the most stable attribute.

? Web Search

When: When you need current information -- latest library versions, recent CVEs, today's news.

Why: Uses the AI's broad knowledge base to answer time-sensitive questions with references.

How: Select Web Search mode -> type your query -> submit.

Demo Input -- paste this to test:

What are the latest breaking changes in Playwright 1.44 and how do they affect existing tests?

Expected: Returns a summary of breaking changes, migration guide, and links to the official changelog.

? Image Generation

When: When you need to describe or prompt an image for design, wireframing, or documentation.

Why: Since direct generation is not always available, returns detailed image prompts for DALL-E/Midjourney.

How: Select Image Generation mode -> describe what you want -> submit.

Demo Input -- paste this to test:

Create a logo for a software testing tool called 'AiMitra'. Modern, dark theme, violet and blue gradient, robot icon.

Expected: Returns a detailed prompt for DALL-E/Stable Diffusion plus design specifications.

6. Dev Tools Workspace

The Dev Tools workspace contains 14 specialised one-shot tools for developers. Each tool has a dedicated UI with options (language, framework, pattern), a focused system prompt, and a copy/download button for the output.

? Chrome Extension

- When:** When you need to generate test code for a live webpage directly from your browser.
- Why:** Reads the actual DOM -- real CSS selectors, XPaths, form structures -- and feeds them to AI. No guessing.
- How:** Install from chrome-extension/ folder -> open any webpage -> click extension icon -> select framework & language -> Generate.

Demo Input -- paste this to test:

Open <https://automationpractice.pl/> in Chrome, click the extension, pick Playwright + Python + POM, click Generate.

Expected: Returns a complete POM class + test file using real selectors from the live page.

? Tool Helper

- When:** When you are not sure which AiMitra tool to use for your task.
- Why:** Guides you to the right tool by asking about your goal and matching it to the best workspace/tool.
- How:** Select Tool Helper -> describe what you want to achieve -> get directed to the right tool.

Demo Input -- paste this to test:

I want to test a REST API that returns user profiles. What tool should I use and how?

Expected: Recommends API Test Generator, explains how to use it, and shows a sample cURL input format.

? Code Explainer

- When:** When you receive unfamiliar code and need to understand it quickly.
- Why:** Breaks down complex code into plain English with line-by-line or block-by-block explanations.
- How:** Paste the code snippet -> click Explain -> choose detail level (summary/detailed/beginner).

Demo Input -- paste this to test:

```
def memoize(func):
    cache = {}
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper
```

Expected: Explains the memoization decorator pattern, closure, and cache dictionary with a real usage example.

?? Code Review

- When:** Before submitting a pull request or reviewing a colleague's code.
- Why:** Reviews for bugs, security issues, performance, readability, and SOLID principles. Gives a score.
- How:** Paste the code -> select language -> click Review. Choose focus area (security/performance/style).

Demo Input -- paste this to test:

```
def get_user(id):
    query = f"SELECT * FROM users WHERE id={id}"
    return db.execute(query)
```

Expected: Flags SQL injection vulnerability, suggests parameterized queries, gives security score 2/10.

? Debug & Fix

- When:** When you have broken code and want the AI to identify and fix the bug automatically.
- Why:** Combines root cause analysis with a working fix. Explains what changed and why.
- How:** Paste the broken code + the error -> click Fix. The AI returns corrected code + explanation.

Demo Input -- paste this to test:


```
async function fetchData() {
  const data = await fetch('/api/users')
  return data.json // missing parentheses
}
```

Expected: Identifies missing () on .json, returns corrected code, explains the difference between property and method call.

? Code Converter

- When:** When migrating code between languages, frameworks, or syntax versions.
- Why:** Accurately converts logic while adapting idioms -- e.g. Python list comprehensions to Java streams.
- How:** Paste source code -> select source language -> select target language -> convert.

Demo Input -- paste this to test:

Convert this Python to JavaScript:

```
users = [u for u in all_users if u['active'] and u['age'] >= 18]
```

Expected: Returns: const users = allUsers.filter(u => u.active && u.age >= 18); with explanation of differences.

? Regex Builder

- When:** When you need to write or understand a regular expression for validation or parsing.
- Why:** Generates regex with explanation, test cases, and code snippet for your chosen language.
- How:** Describe the pattern in plain English -> select language -> build.

Demo Input -- paste this to test:

Regex to validate an Indian mobile number: starts with 6, 7, 8, or 9, exactly 10 digits total.

Expected: Returns: /^[6-9]\d{9}\$/ with explanation, Python/JS usage code, and 5 test cases (pass/fail).

?? SQL Helper

- When:** When writing complex SQL queries, optimising slow queries, or designing schema.
- Why:** Writes optimised SQL with proper JOINS, indexes, window functions, and CTEs. Explains execution plan.
- How:** Describe what data you need in plain English -> select SQL dialect -> generate.

Demo Input -- paste this to test:

Get the top 5 customers by total order value in the last 90 days, include their email and order count. MySQL.

Expected: Returns a CTE-based query with JOIN, GROUP BY, ORDER BY, LIMIT, and an explanation of each clause.

? Git Assistant

- When:** When you need help with git commands, commit messages, branching strategy, or resolving conflicts.
- Why:** Provides exact git commands, explains the effect, and warns about destructive operations.
- How:** Describe your git situation -> get commands and explanation.

Demo Input -- paste this to test:

I accidentally committed my .env file with API keys to main. How do I remove it from git history completely?

Expected: Returns: git filter-branch or BFG Repo Cleaner steps + force push + .gitignore fix + GitHub secret scanning note.

? DevOps Generator

- When:** When setting up CI/CD pipelines, Dockerfiles, GitHub Actions, or Kubernetes manifests.
- Why:** Generates production-ready DevOps configs tailored to your stack and environment.
- How:** Select the DevOps artefact type -> describe your stack -> generate.

Demo Input -- paste this to test:

Generate a GitHub Actions CI/CD pipeline for a React + FastAPI app: lint, test, build Docker image, push to DockerHub, deploy to Render.

Expected: Returns a complete .github/workflows/deploy.yml with jobs, secrets usage, and Docker build/push steps.

? JSON & Mock Data

- When:** When you need sample data for testing, prototyping, or API mocking.
- Why:** Generates realistic, schema-consistent JSON mock data with proper data types and relationships.
- How:** Describe the data structure or paste a schema -> specify count -> generate.

Demo Input -- paste this to test:

Generate 5 mock user records with: id (UUID), name, email, role (admin/user/viewer), createdAt (ISO date), isActive.

Expected: Returns a JSON array of 5 realistic records with proper UUIDs, plausible names, and varied roles.

? BDD Generator

- When:** When writing Gherkin feature files for Cucumber, SpecFlow, Behave, or Pytest-BDD.
- Why:** Creates complete .feature files with Background, Scenario Outlines, Examples tables, and step definition stubs.
- How:** Paste the user story -> select framework (Cucumber/Behave/SpecFlow) -> select scenario coverage -> generate.

Demo Input -- paste this to test:

As a registered user, I want to reset my password via email link, so that I can regain account access. Include edge cases: expired link, already used link, invalid email.

Expected: Returns a .feature file with 5 scenarios + Scenario Outline + step definition stubs in the chosen language.

? API Test Generator

- When:** When writing automated tests for REST APIs using Postman, pytest, or RestAssured.
- Why:** Generates complete API test suites with setup, authentication, status code checks, schema validation, and edge cases.
- How:** Paste the API endpoint details (URL, method, headers, body) -> select framework -> generate.

Demo Input -- paste this to test:

POST /api/auth/login -- body: {email, password} -- returns JWT token. Generate pytest tests for success, wrong password, missing fields, rate limiting.

Expected: Returns a complete pytest file with requests library, fixtures, parametrize, and 4 test functions.

? A11y Checker

- When:** When reviewing a UI for accessibility compliance (WCAG 2.1 AA) before release.
- Why:** Analyses HTML/component code for missing ARIA labels, contrast issues, keyboard navigation, and screen reader support.
- How:** Paste your HTML or React component -> click Check -> get a prioritised list of issues with fixes.

Demo Input -- paste this to test:

```
<button onclick='submit()' style='color:#aaa; background:#bbb'>Submit</button>
```

Expected: Flags: missing type attribute, insufficient colour contrast (3.1:1 vs required 4.5:1), no aria-label, no keyboard focus style.

7. JIRA Tools Workspace

The JIRA workspace contains 7 AI-powered tools designed to make Jira work faster and more consistent. From writing well-structured tickets to generating JQL queries, every tool follows Agile best practices and outputs content ready to copy-paste into Jira.

? Ask Rovo

When: When you have a general Jira/Agile question and want expert guidance.
Why: Acts as a Jira expert -- explains workflows, best practices, configuration options, and Agile ceremonies.
How: Switch to JIRA workspace -> Ask Rovo -> type your question.

Demo Input -- paste this to test:

What is the difference between a Story, Task, and Sub-task in Jira? When should I use each?

Expected: Returns a clear comparison table with use cases and examples for each issue type.

? Ticket Creator

When: When you need to create a well-structured Jira Story or Epic from a rough idea.
Why: Produces a complete ticket with Summary, Description, Acceptance Criteria, Story Points estimate, Labels, and Components.
How: Describe the feature in a few sentences -> click Create Ticket -> copy the output into Jira.

Demo Input -- paste this to test:

We need to add a 'Remember Me' checkbox to the login page. It should keep the user logged in for 30 days using a secure HTTP-only cookie.

Expected: Returns a complete Jira ticket with summary, description, 3 acceptance criteria, 3 story points, and labels.

? Bug Creator

When: When logging a bug found during testing and you need a thorough, reproducible bug report.
Why: Creates a professional bug report with Environment, Steps to Reproduce, Expected vs Actual, Severity, and Attachments note.
How: Describe the bug -> click Create Bug -> copy to Jira.

Demo Input -- paste this to test:

On Safari iOS 17, the checkout button becomes unresponsive after selecting a delivery address. The page does not show any error. Affects 100% of iOS Safari users.

Expected: Returns a bug ticket with severity Critical, browser/OS details, 4 reproduction steps, and suggested fix area.

? JQL Search

When: When you need to find specific issues in Jira using complex criteria.
Why: Generates optimised JQL queries with explanations. Returns alternative queries for edge cases.
How: Describe what issues you want to find -> click Generate JQL -> paste into Jira board filter.

Demo Input -- paste this to test:

Find all critical bugs assigned to me that were created in the last 2 weeks and are still open, sorted by creation date.

Expected: Returns: assignee = currentUser() AND priority = Critical AND issuetype = Bug AND created >= -14d AND status != Done ORDER BY created DESC

? Test Plan Review

When: When you have a test plan in Jira and want an AI review for completeness and quality.
Why: Reviews test plans for missing coverage areas, unclear acceptance criteria, risk gaps, and non-functional requirements.
How: Paste the test plan content -> click Review -> get a structured feedback report.

Demo Input -- paste this to test:

Test Plan: Login feature. Scope: UI testing only. Tests: valid login, invalid login. Exit criteria: all tests pass.

Expected: Flags: missing API test scope, no performance/security testing, vague exit criteria, no negative test for SQL injection.

? Ticket Validator

- When:** Before a ticket enters a sprint -- to check if it is ready (Definition of Ready).
- Why:** Validates against Agile best practices: clear acceptance criteria, estimated, no ambiguous language, dependencies noted.
- How:** Paste the ticket content -> click Validate -> get a readiness score and list of issues.

Demo Input -- paste this to test:

Story: As a user I want a better dashboard. AC: Dashboard should be improved. SP: TBD. Priority: High.

Expected: Score 3/10. Issues: too vague (no specific improvements listed), no story points, acceptance criteria not testable, no wireframe linked.

? Comment Generator

- When:** When you need to add a professional update comment to a Jira ticket.
- Why:** Generates clear, professional comments for status updates, blockers, QA sign-offs, and deployment notes.
- How:** Describe the update situation -> select comment type (status update/blocker/QA pass/deployment) -> generate.

Demo Input -- paste this to test:

QA has completed testing of the login feature. Found 1 minor bug (misaligned button on mobile -- logged as PROJ-456). All critical and major tests passed. Recommending for release.

Expected: Returns a formatted QA sign-off comment with test summary, linked bug, and recommendation.

8. BA Tools Workspace

The BA (Business Analyst) workspace is AiMitra's most unique differentiator -- no other QA/dev AI tool offers a dedicated BA toolkit. It contains 10 tools covering the full BA lifecycle from requirements gathering to stakeholder communication.

? User Story Generator

When: At the start of a sprint when converting stakeholder requirements into development-ready user stories.
Why: Produces stories in standard 'As a... I want... So that...' format with acceptance criteria, story points, and DoD.
How: Select User Story Generator -> choose the user role -> select story type (standard/epic/spike) -> describe the feature.

Demo Input -- paste this to test:

Feature: Users should be able to export their dashboard data as a CSV file. Role: Data Analyst.

Expected: Returns a complete user story with title, narrative, 4 acceptance criteria, 5 story points, and DoD checklist.

? Acceptance Criteria

When: After writing a user story when you need detailed, testable acceptance criteria.
Why: Generates criteria in Gherkin (Given/When/Then), checklist, or both formats -- ready for QA handoff.
How: Paste the user story -> select format (Gherkin/checklist/both) -> generate.

Demo Input -- paste this to test:

As a user, I want to receive an email notification when my order ships, so I can track my delivery.

Expected: Returns 5 Gherkin scenarios covering: order ships, email content, multiple items, cancelled order, unsubscribe.

? Use Case Generator

When: During requirements analysis when you need formal use case documentation for complex features.
Why: Creates complete use cases with Actors, Preconditions, Main Success Scenario, Alternative Flows, and Exception Flows.
How: Describe the feature -> list the actors -> select complexity (simple/standard/detailed) -> generate.

Demo Input -- paste this to test:

Feature: Online appointment booking system. Actors: Patient, Doctor, Receptionist, Notification Service. Complexity: detailed.

Expected: Returns UC-001 with 8-step main flow, 3 alternative flows (reschedule, cancel, no availability), and exception handling.

? Requirements Analyzer

When: When reviewing a requirements document before development begins -- to catch issues early.
Why: Identifies gaps, ambiguities, conflicts, and testability issues. Each finding cites the original text and suggests a fix.
How: Paste the requirements text -> select analysis types (gaps/ambiguities/conflicts/testability) -> analyze.

Demo Input -- paste this to test:

REQ-01: The system shall be fast. REQ-02: Users can upload files. REQ-03: The dashboard should show relevant data.

Expected: Flags: REQ-01 has no measurable metric (suggest: <2s response), REQ-02 has no size/format constraint, REQ-03 is ambiguous.

? Process Flow Generator

When: When documenting a business process for sign-off, training material, or system design.
Why: Creates numbered process flows with decision points (<>), swim lanes by role, and clear Start/End markers.
How: Describe the process -> select AS-IS/TO-BE/Both -> toggle swim lanes -> generate.

Demo Input -- paste this to test:

Document the employee expense reimbursement process: employee submits receipt, manager approves/rejects, finance processes payment, employee receives funds.

Expected: Returns a 4-lane swim-lane flow (Employee/Manager/Finance/System) with 12 numbered steps and 2 decision points.

? BRD Generator

When: When preparing formal project documentation for executive sign-off or vendor procurement.
Why: Generates professional BRD sections with numbered requirements (REQ-001 format), MoSCoW priority, and rationale.
How: Describe the project/feature -> select section (full/scope/functional/non-functional/constraints) -> generate.

Demo Input -- paste this to test:

Project: Customer Self-Service Portal. Customers can view invoices, raise support tickets, and track order status. Section: Full BRD.

Expected: Returns a complete BRD with 15+ numbered requirements, priority levels, constraints, and assumptions section.

? Gap Analysis

- When:** When planning a system migration, process improvement, or digital transformation initiative.
- Why:** Creates a structured gap table (Gap | Category | Impact | Priority | Recommendation) and a phased roadmap.
- How:** Describe current state in first box -> describe desired state in second box -> select category -> analyze.

Demo Input -- paste this to test:

Current: Manual invoice processing via email, 3-day turnaround, 15% error rate.

Desired: Automated invoice portal, same-day processing, <1% error rate.

Expected: Returns 6 identified gaps across Process/Technology/People categories with High/Medium/Low impact and a 3-phase roadmap.

? Stakeholder Update

- When:** After a milestone, incident, or scope change when stakeholders need to be informed professionally.
- Why:** Generates tailored communications -- executive brevity for C-suite, technical detail for dev teams, simple language for end users.
- How:** Describe the update -> select audience (executive/technical/end-users/project-team) -> select tone -> generate.

Demo Input -- paste this to test:

The mobile app launch has been delayed by 2 weeks due to a critical payment gateway integration issue discovered in UAT. New launch date: March 15.

Expected: Returns an executive email with subject line, 3-bullet summary, business impact, and next steps. No technical jargon.

? Meeting Summarizer

- When:** After any meeting -- requirements workshop, sprint review, stakeholder call -- to capture outcomes.
- Why:** Extracts Key Decisions, Action Items (with owner/due date), Discussion Points, and Parking Lot items.
- How:** Paste raw meeting notes (even messy/bullet points) -> select meeting type -> summarize.

Demo Input -- paste this to test:

Discussed login redesign, John said we need SSO, Sarah mentioned deadline is Q2, Mike will check with security team by Friday, old design needs to be archived, budget not confirmed yet.

Expected: Returns structured summary: 1 decision (SSO required), 2 action items (Mike: security review by Friday, Sarah: Q2 deadline confirmed), 1 parking lot (budget).

? Impact Analysis

- When:** Before approving a change request or architectural decision that affects multiple teams or systems.
- Why:** Assesses impact across People, Process, Technology, and Data -- with impact level (High/Medium/Low) and mitigation strategies.
- How:** Describe the proposed change -> select impact areas (people/process/technology/data) -> analyze.

Demo Input -- paste this to test:

We are replacing our MySQL database with MongoDB for the product catalogue service. All existing SQL queries will need rewriting. 3 teams affected.

Expected: Returns impact table: Technology HIGH (query rewrite, ORM change), People MEDIUM (training 3 teams), Process LOW, Data HIGH (schema migration). Risk score: 7/10.

9. Chrome Extension

The AiMitra Chrome Extension brings AI-powered test generation directly into the browser. It reads the live DOM of any webpage and generates complete, runnable test scripts using the actual CSS selectors, XPaths, and form structures found on the page -- no generic placeholders.

Features

- * DOM analysis: captures forms, inputs, buttons, links, ARIA roles, data-testid attributes
- * Element Picker: click any element on the page to add it to the test target list
- * 4 frameworks: Playwright, Selenium, Cypress, WebdriverIO
- * 4 languages: Python, JavaScript, TypeScript, Java
- * 3 patterns: POM (Page Object Model), BDD/Gherkin, Simple function-based
- * 13 AI providers configurable -- same providers as the main app
- * Auto-injects content script into pages opened before extension was loaded

Installation

- * Open Chrome and navigate to `chrome://extensions/`
- * Enable 'Developer Mode' (toggle, top right)
- * Click 'Load Unpacked'
- * Select the `chrome-extension/` folder from the AiMitra project
- * The AiMitra icon appears in the Chrome toolbar
- * Click the icon on any webpage to open the generator

Demo -- Test it now

Step 1: Open any website

Navigate to: <https://automationpractice.pl/index.php?controller=authentication>

Step 2: Click extension icon

The popup shows: Forms: 1, Buttons: 2, Inputs: 2, Links: 8

Step 3: Configure and generate

Framework: Playwright | Pattern: POM | Language: Python -> Click Generate Test Code

Expected: A complete Python POM class with EmailInput, PasswordInput, SignInButton locators, plus a test file with login happy path and negative test cases.

10. AI Providers & Models

Provider	Tier	Key Models	Best For
Google Gemini	Free	gemini-2.0-flash, gemini-1.5-flash, gemma-3-27b	Best free option for vision and large context

Groq

11. Architecture & Data Flow

AiMitra uses a clean client-server architecture where the React frontend handles all UI state and the FastAPI backend acts as a secure proxy between the user and AI provider APIs.

Request Flow -- Non-Streaming

1. User types a message and clicks Send
2. React sends POST /api/chat with: message, mode, api_key, model, provider, temperature, history
3. FastAPI validates the request (Pydantic ChatRequest model)
4. prompt_router.py maps the mode to a (system_prompt, user_message) pair
5. _pick_service() selects the correct backend service based on provider/model
6. The service calls the AI provider's API (httpx async call)
7. Response text is returned to FastAPI -> JSON response to React -> rendered as markdown

Request Flow -- Streaming (SSE)

When streaming is enabled (default), the same request goes to POST /api/chat/stream. FastAPI returns a StreamingResponse wrapping an async generator. Each AI token is wrapped as 'data: {"chunk": "..."}\n\n' and sent via SSE. React's native Fetch API reads the ReadableStream line-by-line and appends each chunk to the message in real time. The UI shows the message growing token by token.

Security Design

- * API keys are stored in browser localStorage -- never on the server or in any database
- * Keys are forwarded per-request in memory only -- not logged, not cached server-side
- * API key patterns are scrubbed from all error messages before returning to the frontend
- * CORS is open (*) for development -- lock down origin in production deployment
- * No user accounts required -- zero PII collected

Deployment

- * Single-service deploy: FastAPI serves the built React app from backend/static/ (npm run build -> copy to static/)
- * Free hosting: Render.com, Railway.app (backend + static), Vercel (frontend only)
- * Docker: Dockerfile included -- docker build -t aimitra . && docker run -p 8000:8000 aimitra
- * Environment: Set REACT_APP_API_URL for production backend URL in frontend .env

12. About the Developer

Shiv Kant Kumar

Software Engineer | QA Enthusiast | AI Tools Builder

Connect on LinkedIn:

<https://www.linkedin.com/in/shivkantkumar/>

AiMitra was built to solve a real problem: QA engineers, BAs, and developers need domain-expert AI tools, not generic chatbots. Every tool was designed from daily work experience.

